

Tokens and Orders: Understanding Generative Control Policies in Discretized Action Spaces

Sriram Kutty, William Gong, Anshul Shah

University of Washington

{skutty, wgong2, anshul7}@uw.edu

Abstract

*Generative control policies for robot learning have converged on two dominant paradigms: **continuous diffusion** policies and **autoregressive (AR)** policies over discretized action tokens. While both achieve strong empirical performance, prior comparisons conflate multiple design axes: the action representation, model architecture, and training objective. This makes it difficult to attribute performance differences to the sequence modeling paradigm itself. We address this by placing AR and masked diffusion (MD) policies within a common discretized action space, using a shared transformer backbone and tokenizer to isolate the effect of the underlying training and decoding procedure. Through this controlled setting, we systematically study three tokenization strategies. Across all tokenizers, MD consistently outperforms AR. To understand why, we analyze MD’s reverse diffusion process and find that it learns an adaptive decoding order that differs meaningfully from AR’s fixed left-to-right generation. These results offer a cleaner characterization of when and why iterative refinement outperforms causal generation in discretized robot control.*

1. Introduction

Behavior cloning policies for control settings increasingly predict **action chunks** rather than individual actions [1], improving temporal consistency and reducing compounding error. Among action-chunking policy representations, two generative control policy paradigms have emerged as dominant [2, 8]. **Continuous diffusion** policies model action chunks through iterative denoising in continuous spaces, while **autoregressive (AR)** policies generate discretized action tokens sequentially. Both approaches achieve strong empirical performance and naturally represent multimodal action distributions.

Despite their success, it remains unclear which properties of these paradigms are fundamentally responsible for

their performance. Existing comparisons confound multiple factors simultaneously. Both can represent multimodal action distributions, but don’t cleanly compare. Firstly, AR operates on a discretized action space whereas DDPM-based diffusion policies [2, 3] operate on continuous action spaces. Secondly, to our knowledge the techniques haven’t been compared on identical model architectures. Therefore, these policies differ not only in their training objectives and inference procedures, but also in action representation, output parameterization, and model architecture. Improvements attributed to diffusion or autoregression may therefore arise from these auxiliary design choices rather than the sequence modeling paradigm itself.

To address this issue, we instead study both paradigms within a common discretized action space. Using a shared tokenizer and transformer backbone, AR and discrete diffusion policies possess comparable representational capacity and model identical multimodal token distributions. This setting isolates the effect of the underlying training and decoding procedure while minimizing architectural confounds.

Beyond enabling a controlled comparison, discretized control introduces a largely unexplored setting for diffusion models. While masked discrete diffusion has recently demonstrated promise in language generation [4, 5], relatively little is understood about its behavior when applied to robot control. Along the way, we created new techniques and documented the unique challenges of training discrete diffusion models, which are understudied for control.

This work has two complementary goals:

1. We seek to understand the algorithmic differences between autoregressive and diffusion-based sequence modeling under an apples-to-apples experimental setting.
2. We investigate which capabilities unique to discrete diffusion contribute meaningfully to policy performance in robot control.

2. Preliminaries

2.1. Common Policy Architecture

To isolate differences arising from the sequence modeling paradigm rather than architectural choices, all policies share an identical transformer backbone and action tokenizer whenever possible. Policies consume identical observation representations and predict fixed-length action chunks. The only difference between methods lies in the training objective and decoding procedure.

By holding architecture, tokenizer, and action representation fixed, observed performance differences can be attributed primarily to autoregressive versus diffusion training and inference.

2.2. Model Types

Autoregressive (AR) policies model the joint distribution over action tokens through causal factorization, predicting one token conditioned on previously generated tokens. In our implementation we condition generation on state s which has a learnable encoding to the model’s latent dimension d_{model} (we generally used 72). Both at training and inference, sinusoidal positional embeddings are added. To train action token prediction, we use logit shifting and next token prediction with cross entropy loss; for each expert tokenized action sequence, our loss function is

$$-\frac{1}{L} \sum_{i=0}^L \log p_{\theta}(a_i | s, a_{<i})$$

At inference, the model is given state s and samples from the predicted next token’s categorical distribution generated by the softmaxed logits scaled by temperature.

Masked diffusion (MD) policies instead learn to reconstruct partially masked action chunks through denoising and generate trajectories through iterative reverse diffusion. Here, we use a transformer with bidirectional attention to allow for the chunk to be refined as whole globally. At training, for each action chunk of size L tokens, a t is sampled from a categorical distribution over $[1/L, 2/L, \dots, L/L]$. This is a uniform distribution, but in practice we use a parameter q to upweight the probability of L/L as extra training on fully masked chunks improves performance. Then, independently, each token in the action chunk is masked with token [MASK] with probability t . t is converted to a sinusoidal embedding of size d_{model} then through a two layer learned MLP. The final time embedding of d_{model} , as well as the sinusoidal position embedding is added to each token’s embedding both at training and inference. During training, the loss is calculated exactly the same as in AR, but only on masked tokens.

At inference, we follow a confidence based iterative revealing process [6]. The action chunk of size L tokens are

all initialized with [MASK]. The value t is defined at any step as the fraction of the L tokens currently masked. Conditioned on s , incorporating the t and positional embeddings, we generate categorical distributions for each token position. For each currently masked position i , we sample a token s_i from its distribution. Let $p_i(s_i)$ be the probability of sampling s_i from the categorical distribution at i . Then we choose the highest sampled confidence/probability position to reveal $\arg \max_i p_i(s_i)$, leaving the rest masked. We do not remask already revealed tokens, although we explore this later in Section 6. From now on, all mentions of MD-pin or generic mentions of MD refer to this inference protocol, unless otherwise specified.

2.3. Training and Evaluation Protocol

All experiments are conducted on the D3IL pushing benchmark [10] using identical optimization hyperparameters and training schedules across methods. Policies are trained for 100 epochs using the default optimization settings of the released implementation.

Unless otherwise mentioned, all models are evaluated on 8 rollouts for each of the 30 contexts of the D3IL Pushing task.

3. Impacts of Discretization on Policy Learning

To achieve multimodality in transformer architectures, we discretize the action space into tokens, with outputs of any type of model in the form of categorical distributions over the vocabulary. The simplest form of discretization is binning. For each action dimension d , we linearly normalize the training data to $[-1, 1]$ and linearly space k number of bins within that range; thus there are $k \times d$ total tokens. As the authors of FAST note, naive binning performs with autoregressive models in simple environments on data sampled at a low frequency [8]. Our AR policy had modest performance as seen in Figure 1, but MD failed to produce results.

The **FAST tokenizer** [8] was introduced as an improvement to naive binning, specifically designed for autoregressive models. It is built on the assumption that for action chunks, the frequency space is less correlative than the time space, and low frequencies are more important than high (ex: jitter) for control. FAST performs a discrete cosine transform (DCT) for each dimension over the training action dataset, and quantizes the outputs to create a vocabulary. Tokens of an action chunk are put in frequency-major order: the lowest-frequency coefficient across all action dimensions comes first, followed by the second-lowest frequency across all dimensions, and so on. It further does byte-pair encoding (BPE) until a fixed vocabulary size is reached. The ordering of lower to higher frequency is meant to help autoregressive models as lower information, fine

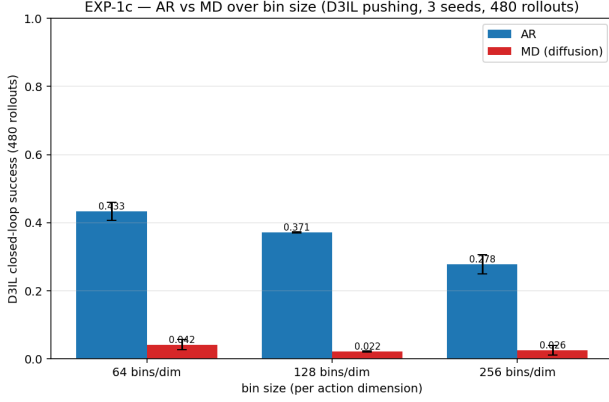


Figure 1. Performance of AR and MD on naive binning across different bin counts

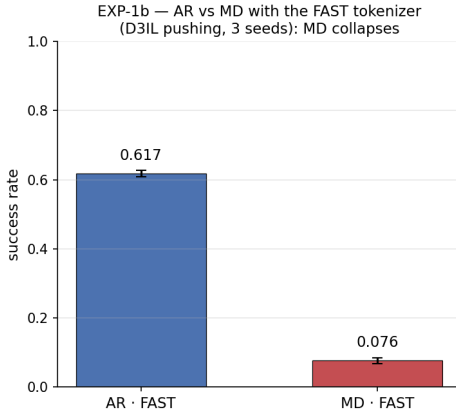


Figure 2. Performance of FAST

grained control tokens are later and thus their generation is conditioned upon higher information coarse action tokens.

Models generating sequences of FAST tokens, due to variable length / information content as a result of BPE, may often not decode to $H \times d$ chunks. Additionally, diffusion models may have more difficulty than AR learning end of sequence tokens [EOS]. With bidirectional attention on a fixed length token canvas, they must resolve noisy signals from tokens past the true [EOS]. We see this reflect in the performance in Figure 2.

It’s been suggested that diffusion is spectral autoregression [9]. That suggests diffusion with naive binning in the time domain may perform as well as FAST in AR. We did not see these results in Figures 1 and 2, but there are confounding factors (including MD was not remasked, emulating standard DDPM and continuous diffusion), so we cannot make a firm conclusion.

The **OAT tokenizer** [12] was introduced, again designed for autoregressive models. Like FAST, it attempts to also order coarser, more information dense tokens first to work

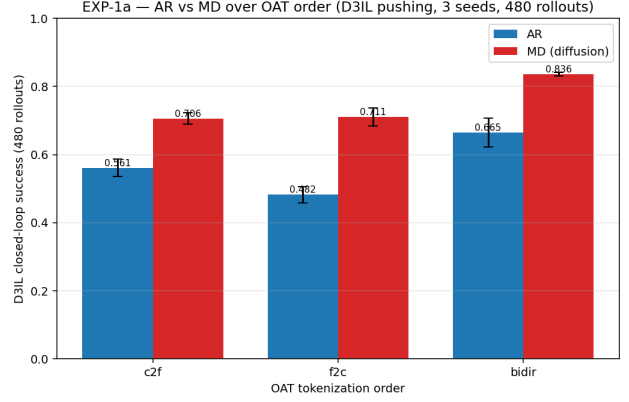


Figure 3. MD and AR policies with standard, coarse-to-fine (c2f) OAT, standard OAT but reversed token order in training data (f2c), and our bidirectional OAT.

well with causal autoregressive generation. However it generates fixed length tokens L for an action chunk that always decode into $H \times d$ actions, where H is the number of actions / horizon length in a chunk and d is the dimension of actions. To train OAT, action chunks are concatenated with L learnable registers in a transformer. The registers causally self-attend to each other, and cross attend with a causal transformer decoder to regenerate continuous action chunks, and trained on reconstruction error. Registers are turned into a discrete vocabulary with FSQ [7]. OAT is further trained to favor autoregressive coarse to fine generation by randomly dropping out a suffix of registers before decoding, forcing earlier tokens to have enough information to decode the full action chunk.

It’s interesting to note that when reversing the order of OAT tokens so that training data action chunks are now fine to coarse, performance did not degrade substantially for AR models (see Figure 3). We hypothesize that since the most important coarse tokens are still conditioned upon the state information, they will still be predicted properly. Only “fine” tokens will suffer as they likely require information about the broader action, not just the state, to be properly generated. MD models had no meaningful differences, which follows as training token order should not effect a bidirectional model.

OAT outperforms FAST for AR models and MD models even them, perhaps owing to a fixed chunk length for all without decode errors.

We were inspired how OAT was derived from first principles to favor AR and did the same for MD. We created **bi-directional OAT** with the idea that each token / position should have equal information content and thus predictive power. Thus with any token generation order, regardless of the current subset of token positions revealed, it should work equally well; in contrast with MD on stan-

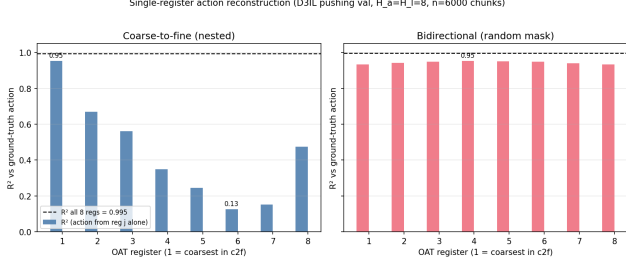


Figure 4. Standard and bidirectional OAT reconstruction error of a full ground-truth action chunk from only a single register input

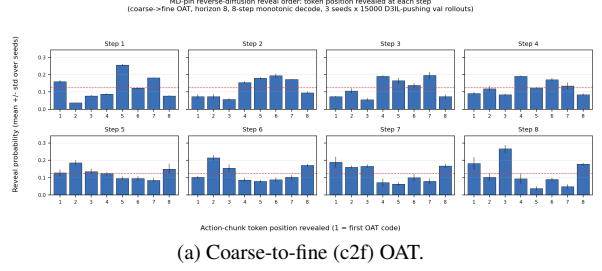
standard OAT, it’s possible to generate high information tokens conditioned on lesser information ones. Bidirectional OAT is setup the same for training except with bidirectional attention amongst the registers and rather than suffix dropout, random masking to spread predictive power across all token positions. Like the authors of OAT [12] who ablated suffix dropout, we ablated random masking and found it boosted performance.

Bidirectional OAT boosted the performance of both models, surprisingly for AR models too. However as the authors noted, when the token length $L \geq H$ the horizon length, as was done in most of our experiments $L = H = 8$, there isn’t much of an information bottleneck. As seen in Figure 4, bidirectional OAT has high predictive power in all tokens, just as good as the predictive power of the first token in coarse-to-fine (c2f) tokenization. So it is understandable that AR performs better on bidirectional OAT where all token positions are about or more informative as the original.

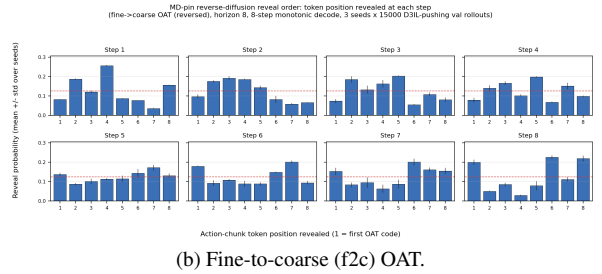
4. Adaptive Decode Orders

We were fascinated by how consistently MD outperformed AR across each of the unique OAT tokenizers – even using the c2f tokenizer built specifically to empower the primary inductive bias of AR training algorithms. To investigate this further, we interpret the reverse diffusion process of MD by trying to explicitly grasp the order in which tokens were denoised. We accomplish this by training MD models using each of the OAT tokenization schemes and comparing the order in which they reveal tokens. Specifically, Figures 5a, 5b, and 5c stitch histograms for each denoising step plotting the average rate that the model unmask a token at that step against the token positions.

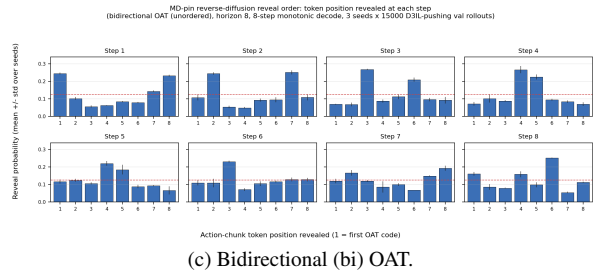
Comparing the three histograms reveals how the learned decoding strategy changes as a function of the underlying tokenization. Firstly, for the c2f OAT tokenization scheme, the trained MD model learns a unique order that differs significantly from the left-to-right (token 1 to token 8) generation that AR is forced to conduct. We initially hypothesized that left-to-right generation should be the upper bound for action chunk generation for a c2f tokenization scheme.



(a) Coarse-to-fine (c2f) OAT.



(b) Fine-to-coarse (f2c) OAT.



(c) Bidirectional (bi) OAT.

Figure 5. Reverse diffusion decode-order histograms for different tokenization strategies. Each row corresponds to a different tokenizer and shows the frequency with which each action token is selected for decoding at each reverse diffusion timestep. Each figure plots the reverse diffusion process across 3 different MD models trained on the D3IL pushing task.

Given that OAT’s hierarchical encoding algorithm naturally encodes most high-level information about the trajectory into the first few tokens, why wouldn’t it be best to generate tokens from “coarse” to “fine”? We conjecture that because the “fine” tokens generated near the end of the sequence store less information they are similar across most action chunks possible to generate, whereas “coarse” tokens can heavily differ. AR models trained through causal masking and logit shifts are forced to *select the most important token first* conditioned only on information about the state. However, MD models tend to place higher probability mass on revealing middle tokens first and then revealing “coarse” tokens towards the end (see Figure 5a).

Crucially, by our hypothesis, it should be *easier* to select middle or more fine-grained tokens first and condition on those tokens to reveal tokens that contain the most information about the trajectory last. We can interpret this comparison through the following relationship: AR models are

burdened with the responsibility to commit to a trajectory within the first step, but MD models have the slack to conservatively fill in details about the trajectory before finally committing to a trajectory.

Additionally, observe that leveraging f2c OAT tokenization leads to histograms flipped across the probability axis for each of the denoising steps (see Figure 5b). This confirms MD’s discovery of robust denoising orders.

The denoising orders in expectation converge to the plots in Figure 5, however, each of the leveraged action chunks are generated conditioned on different states entered during policy rollouts. Is there a generalizable, state-independent decoding pattern that MD learns? Since the D3IL Pushing task has a basic state-space governed by a 2D Cartesian plane, we attempt to determine if MD’s reverse diffusion process is state-independent. Specifically in Figure 6, we sample states arbitrarily from the dataset and for each state, visualize the Jensen-Shannon divergence between the global probability distribution over reveal events and the distribution over reveal events induced by entering that state multiple times. We observe that the divergence stays fairly close to 0 for most of the states.

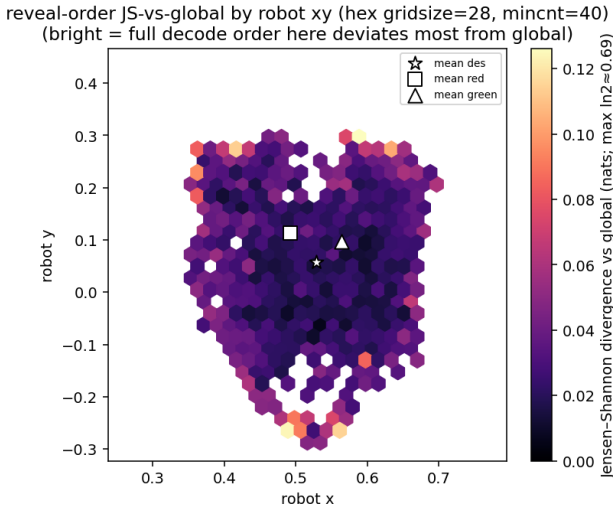


Figure 6. Jensen-Shannon divergence between the global reveal-event distribution and state-conditioned reveal-event distributions for randomly sampled states in the D3IL Pushing task. The consistently low divergence across most states suggests that MD learns a largely state-independent reverse diffusion decoding pattern.

5. Masked Diffusion

5.1. Taxonomy of Masked Diffusion Capabilities

Our usage of MD models in the discretized action space regime motivates investigation into the specific properties of MD policies that lead to robust performance in control settings. Additionally, we seek to understand which of these

capabilities can be extrapolated to continuous diffusion policies (the state-of-the-art for real-robot behavior cloning).

In Section 5, we detail how MD policies learn a unique decoding order that benefits action chunk generation at inference-time. Furthermore, we highlight **learnable decoding orders** as an important capability of MD policies.

Prior works [2, 3] have highlighted that the reverse diffusion process of continuous diffusion policies enable **self-correction** – in particular, the generation output at each denoising step can globally refine the output from previous time-steps. This leads to emergent properties such as adherence to the data manifold of the demonstrations dataset [13]. We observe that it is possible to inject this property into our control MD policies by modifying the reverse diffusion process to enable re-masking previously revealed tokens. Stochastically adding masks to all tokens in a sequence allows for self-correction and a much clearer analogue to continuous diffusion policies arises. Alternatively, if we can’t mask previously revealed tokens, the model can no longer change those tokens and the policy’s generation of the remaining tokens will forever condition on previously revealed tokens in the sequence.

Both the properties of (1) learnable decoding orders and (2) self-correction are entangled and we seek to rigorously demonstrate how these properties impact performance. Thus, we establish a set of 4 unique classes of reverse diffusion processes to rigorously ablate these properties:

- A. **Confidence-based pinning (MD-pin):** Confidence-based pinning is the standard MDLM sampler [5] that for each denoising timestep, unmask the individual token that the policy has the highest confidence score on and masks the remaining tokens that haven’t already been revealed. Crucially, revealed tokens are kept and can *never* be re-masked in future denoising steps.
- B. **Confidence-based re-masking:** Confidence-based re-masking follows the same confidence-based denoising schedule as confidence-based pinning, but allows revealed tokens to be re-masked. At each timestep, every revealed token is independently re-masked with some probability, where tokens the model was *least* confident about when they were last decoded are the most likely to be sent back to [MASK]. To accomplish this, we implement the ReMDM sampler from Wang et al. [11]. This lets the model revisit and correct tokens it is unsure about across denoising steps, rather than locking them in permanently.
- C. **Uniform pinning:** At any timestep, for all of the tokens that are still [MASK] tokens in the target sequence, uniform pinning uniformly selects one among them to reveal.

D. **Uniform re-masking:** At timestep t , uniform re-masking reveals or re-mask each token in the sequence based on a Bernoulli random variable with probability $\frac{t}{T}$ where T is the total number of timesteps. With probability $\frac{t}{T}$, the token is revealed and with probability $\frac{T-t}{T}$, the token is re-masked.

The factorial design enables the following controlled comparisons:

- **Self-correction ablation:** Compare $(A) + (C)$ against $(B) + (D)$. The only difference is whether previously decoded tokens may be revisited, isolating the effect of iterative self-correction.
- **Decode-order ablation:** Compare $(A) + (B)$ against $(C) + (D)$. The expected remask budget is matched across policies, so the only difference is whether re-masking is confidence-guided or timestep-driven, isolating the benefit of adaptive decode ordering.

Table 1. Factorial ablation of the two capabilities unique to masked diffusion on the D3IL Pushing task. The same coarse-to-fine grained OAT tokenizer was consistently used across all policies for action space discretization. Entries report mean success rate with 95% confidence intervals over three seeds.

Decode Order	Self-Correction	
	Enabled	Disabled
Timestep-based	0.590 ± 0.050	0.593 ± 0.071
Confidence-based	0.653 ± 0.025	0.707 ± 0.024

From Table 1, the decode-order ablation reveals that using a confidence-based reverse diffusion process leads to significant improvements in success rate for our experimental setup. This corroborates results found in the previous section.

The results displayed in Table 1 suggest that on our given setup and for our implementation, enabling re-masking can worsen performance. We hypothesize that for our experimental setup, the property of self-correction does not meaningfully help our policy at all because previously revealed tokens can “leak” information into tokens generated in future time-steps leading to similar generation schemes. This can be true regardless of whether re-masking is enabled or not. We leave a broader study on the importance of self-correction in discrete diffusion across several control tasks and longer horizon generation settings to future work.

5.2. Training Dynamics

In practice, we found altering the data distribution to increase the number of fully masked training sequences for

discrete diffusion models was pivotal for success, so there are other factors. However, there is not a clean analogue for the noise distribution for AR and the total amount of data is still the same.

Our hypothesis is that this is correlated with how the first tokens that are denoised from a completely masked chunk of tokens affect the remainder of tokens selected for the rest of the reverse diffusion process. Specifically, after the first few tokens are denoised at $t = 0$, the MD policy conditions on the partially unmasked input to determine its outputs at $t = 1$.

For MD with confidence pinning in particular, where tokens that are denoised in the reverse diffusion process are never re-masked, the selection of the first token can largely determine the remainder of the denoising steps.

6. Limitations + Future Work

One significant limitation was the lack of diversity of environments. To have more generalizable results, we’d like to test on more than just D3IL’s Pushing task. A more general limitation was that this study was motivated by understanding the properties of continuous diffusion by porting it on to the discrete space. However, masked diffusion isn’t a perfect analog of continuous diffusion, especially given the binary nature of refinement: masked tokens don’t gain information during inference until they are chosen to be revealed, and once revealed a token isn’t further refined (MD-pin) or if so, requires re-masking and its progress lost. This study cannot really extrapolate to continuous diffusion, but we did gain more perspective on what general aspects of iterative refinement processes of autoregressive and diffusion based models are helpful.

Our self correction results should not necessarily taken at face value, it was a limited study with tokenized action chunks with often redundant information. On the topic of redundant information, it would be interesting to analyze bidirectional OAT when there is truly an information bottleneck, where each token is equally important but orthogonal. More rigorous theory and controlled experiments should be done on the decode order, especially as we don’t fully understand why predicting lesser information states first before the more informative ones improves performance rather than doing c2f AR generation. We explored self conditioning, which has shown to be fruitful for diffusion models, but ultimately didn’t implement it in our final runs.

References

- [1] T. Z. Zhao *et al.*, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023. 1

- [2] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025. 1, 5
- [3] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 1, 5
- [4] J. Austin *et al.*, “Structured denoising diffusion models in discrete state-spaces,” in *Advances in Neural Information Processing Systems*, vol. 34, pp. 17981–17993, 2021. 1
- [5] S. S. Sahoo *et al.*, “Simple and effective masked diffusion language models,” in *Advances in Neural Information Processing Systems*, vol. 37, pp. 130136–130184, 2024. 1, 5
- [6] H. Chang *et al.*, “MaskGIT: Masked generative image transformer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2
- [7] F. Mentzer *et al.*, “Finite scalar quantization: VQ-VAE made simple,” in *International Conference on Learning Representations (ICLR)*, 2024. 3
- [8] K. Pertsch *et al.*, “FAST: Efficient action tokenization for vision-language-action models,” *arXiv preprint arXiv:2501.09747*, 2025. 1, 2
- [9] Sander Dieleman. *Diffusion is spectral autoregression*. Sander AI Blog, 2024. Available at: <https://sander.ai/2024/09/02/spectral-autoregression.html>. 3
- [10] X. Jia *et al.*, “Towards diverse behaviors: A benchmark for imitation learning with human demonstrations,” *arXiv preprint arXiv:2402.14606*, 2024. 2
- [11] R. Yu, Q. Li, and X. Wang, “Discrete diffusion in large language and multimodal models: A survey,” *arXiv preprint arXiv:2506.13759*, 2025. 5
- [12] C. Liu *et al.*, “OAT: Ordered action tokenization,” *arXiv preprint arXiv:2602.04215*, 2026. 3, 4
- [13] C. Pan, G. Anantharaman, N.-C. Huang, C. Jin, D. Pfommer, C. Yuan, F. Permenter, G. Qu, N. Boffi, G. Shi, and M. Simchowitz, “Much Ado About Noising: Dispelling the Myths of Generative Robotic Control,” *arXiv preprint arXiv:2512.01809*, 2025. 5